RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT - 5

Prepared by: Mohamed B Sirajuddeen

Reg No : 22TD0053

Subject Name : Embedded Systems

Subject Code : CST62

Year/Sem/Sec : Y3/S6/B

Date of Submission: 13/05/2025

Unit : 5

Teacher's Sign :

Department Vision And Mission

Vision

Cultivate student as technocrats, researchers and entrepreneurs in the field of computer science and engineering.

Mission

→ To impart quality education in computer science and Engineering, by means of learning techniques and value-added courses.

→ To inculcate professional excellence and research culture by encouraging projects in cutting-edge technologies through industry interaction.

→ To build leadership skills, ethical values and teamwork among the students.

→ To strengthen the collaboration of department and industry through internships, mentorships and professional body activities.

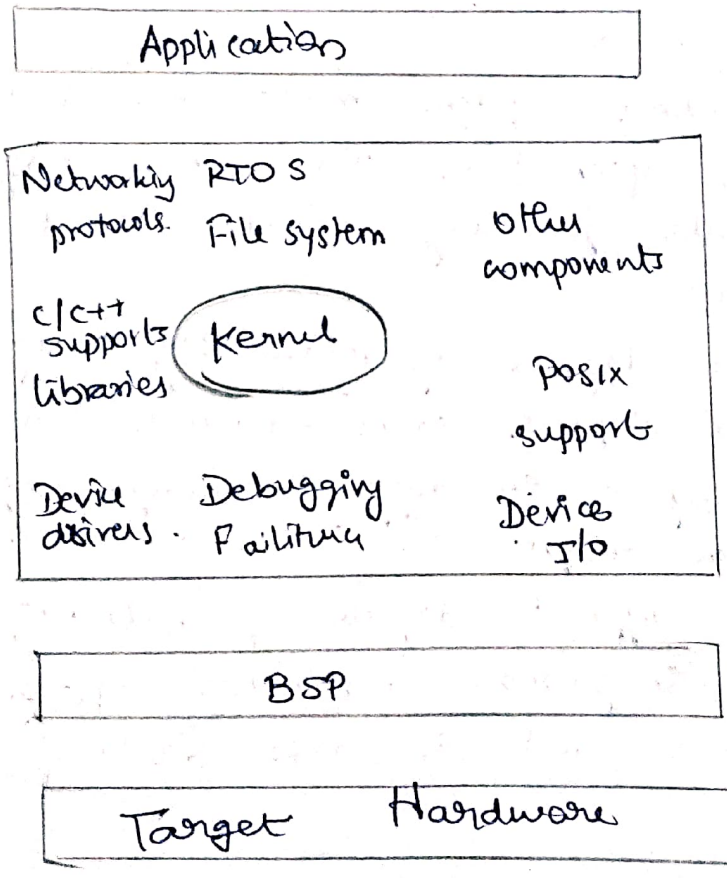
1) Define RTOS and the characteristics of RTOS

RTOS

A real time operating system (RTOS) is a program that schedules execution in a timely manner, manages system resources, and provides a consistent foundation for developing application code.

Application code designed on an RTOS can be quite diverse ranging from a simple application for a digital stopwatch, to a much more complex application for aircraft navigation.

For eg. in some applications an RTOS comprises only a kernel which is the core supervisory software that provides minimal logic, scheduling and resource management algorithms.



Although many RTOS can scale up or down to meet application requirements, this book focuses on the common elements at the heart of all RTOS es - the Kernel. Most RTOS kernels contain the components.

Characteristics of an RTOS

An application's requirements define the requirements of its underlying RTOS. Some of the more common attributes are:

- Reliability
- Predictability
- Performance
- Compactness
- Scalability

→ These attributes are discussed next - however the RTOS attributes needs depends on the type application being built

Reliability

→ Embedded system must be reliable depending on the application, the system might need to operate for long period without human interventions.

→ Different degrees of reliability may be required. For eg. a digital sales powered calculations might reset itself if it does not enough light, yet the calculation might still be considered acceptable.

→ Although different degrees of reliability might be acceptable in general a reliable system is one that is available and does not fail.

Predictability

→ Because many embedded systems are also real time systems meeting time requirements is key to ensuring proper operation.
The RTOS used in this case needs to be predictable to a certain degree.

→ The term determines describes RTOS with predictable behaviour in which the completion of operating system calls occur within known time-frames.
→ Developers can write simple benchmark programs to validate the determinism of an RTOS. The result is based on timed responses to specific RTOS calls.
→ In a good deterministic RTOS, the variance of the response times for each type of system call is very small.
→ a temperature value from a sensor.
→ a bitmap to draw on a display.
→ a keyboard event.
→ a text message to print to an LCD
→ a data packet to send over the network.

Message queue storage

→ System pools using a system pool can be advantageous if it is certain that all message queues until never be filled to capacity at the same time.
The advantage occurs because system pool typically save on memory use.
The downside is that a message queue with large memory not leaving enough memory for other message queue.

Private buffers Using private buffers on the other hand requires enough reserved memory area for the full capacity of every message queue that will be created.

Operation and use

- Creating and deleting message queues.
- → Sending and receiving messages and
- ⇒ obtaining message queue information.

3) Define semaphore and it's operation 2use

Semaphores

A semaphore is a kernel object that one or more threads of execution can acquiring or release for the purpose of synchronization or or mutual exclusion.

→ When a semaphore is first created the kernel assigns to it an associated semaphore control block (SCB) a unique ID, a value (binary or count) and a task waiting list

Semaphore operations and use:

- → Creating and deleting semaphores.
- → Acquiring and releasing semaphores.
- → Clearing a semaphore, a task waiting list and
- → getting semaphore information.

+ Binary specify the initial semaphore state and the task waiting order.

+ Counting Specify the initial semaphore count and the task waiting order.

- Do not wait task makes a request to acquire a semaphore token but if one is not available the task does not block.
  - → wait and signal synchronization.
  - → credit tracking synchronization.

- multiple tasks wait and signal synchronization
- single shared resources access synchronization.
- Recursive shared resources access synchronization
- multiple shared resources access synchronization

→ A semaphore is like a key that allows a task to carry out some operation or to access a resource. If the task can acquire the semaphore, it can carry out the intended operation or access the resource.

→ The kernel tracks the number of times a semaphore has been acquired or released by maintaining a token count.

→ The task waiting list tracks all tasks blocked while waiting on an unavailable semaphore. These blocked tasks are kept in the task waiting list in their first in / first out (FIFO) order or highest priority first order.

→ A kernel can support many different types of semaphore including binary counting and mutual exclusion semaphore.

→ The kernel move this unblocked task either to the running state if it is the highest priority task or to the ready state, until it becomes the highest priority task and is able to run.

3) Defining message queue, states, contents, storage operations and uses.

Message queue

→ A message queue is a buffer like object through which tasks and ISR send and receive message

to communicate and synchronize with data.

→ A message queue is like a pipeline. It temporarily holds messages from a sender until the intended receive is ready to read them.

Message queue states

→ When a message queue is first created the FSM is in the empty state. If a task attempts to receive messages from this message queue while the queue is empty the task block and if it chooses to, is held on the message queue's task waiting list, in either FIFO or priority based order.

Message queue content

Some of the messages can be quite long and may exceed the maximum message length, which is determined when is determind when the queue is created,

Performance

This requirement dictates how an embedded system must perform fast enough to fullfill its timing requirements.

Compactness

Because RTOS's can be used in wide variety of embedded systems, they must be able to scale up or down to meet application specific requirements

Scalability

Application design constraints and cost constraints help determine how compact an embedded system can be.
These design requirements limit system memory which in turn limits the size of the application and operating system.